



School of Future Tech

Case Study Report

DATA STRUCTURES AND ALGORITHMS PROJECT REPORT

Distributed Memory Grid Shard Cluster

Problem Statement No.: 84

Submitted By:

Nandeni Tiwari

Roll no: 150096725033

University:

ITM Skills University

Subject:

Data Structures and Algorithms

Academic Year:

2025–2026

PROJECT TITLE

Distributed Memory Grid Shard Cluster

A C++ based simulation of a distributed storage system that demonstrates the practical implementation of various Data Structures and Algorithms including Hashing, Stack, Queue, Binary Search, Merge Sort, Graphs, Dijkstra's Algorithm, and Priority Queue.

PROBLEM STATEMENT

MemShard is a high-speed distributed storage cluster that distributes memory access keys across multiple nodes and replicates data for fault tolerance. The system faces challenges in memory key management, request processing, recovery tracking, node verification, network representation, shortest path determination, and storage optimization.

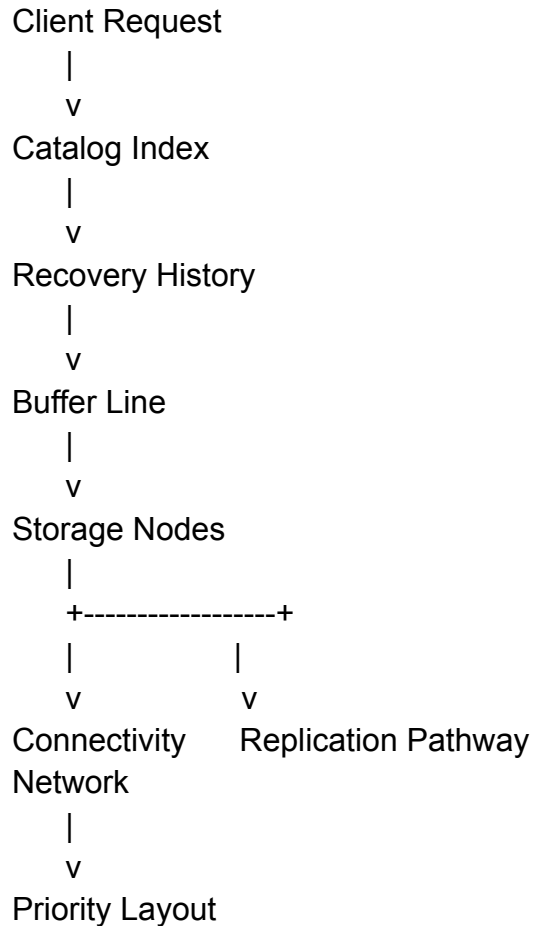
The objective of this project is to design and implement a Distributed Memory Grid Shard Cluster using C++ and appropriate Data Structures and Algorithms. The system simulates real-world distributed storage concepts similar to Apache Cassandra and Hazelcast while demonstrating efficient storage management techniques.

OBJECTIVES

- To simulate a distributed memory storage environment.
- To distribute and manage memory keys efficiently.
- To track write operations for recovery purposes.
- To process client requests in arrival order.
- To verify node membership efficiently.
- To sort data blocks based on compression benefit.
- To represent cluster connectivity using graphs.
- To determine minimum latency replication routes.
- To manage storage space through intelligent eviction.
- To demonstrate practical applications of Data Structures and Algorithms.

DESIGN / ARCHITECTURE

Architecture Diagram



Architecture Description

The Distributed Memory Grid Shard Cluster consists of eight modules, each representing a specific operation within a distributed storage environment.

The Catalog Index stores memory keys and node mappings. Recovery History tracks write operations for fault recovery. Buffer Line manages incoming requests. Registry Index verifies node membership. Line Sorter ranks data blocks according to compression benefit. Connectivity Network models node connections. Replication Pathway determines the shortest latency path between nodes. Priority Layout manages storage optimization by removing old data blocks.

Together these modules simulate the functionality of a distributed storage system.

ALGORITHM DESCRIPTION

Catalog Index

DSA Used: Hashing

Stores memory keys and maps them to storage nodes for fast insertion and retrieval.

Recovery History

DSA Used: Stack

Maintains write operations using the LIFO principle, allowing recent operations to be recovered first.

Buffer Line

DSA Used: Queue

Processes incoming requests using FIFO order.

Registry Index

DSA Used: Binary Search

Verifies whether a node exists within a sorted node registry.

Line Sorter

DSA Used: Merge Sort

Ranks data blocks based on compression benefit using efficient sorting.

Connectivity Network

DSA Used: Graph (Adjacency List)

Represents connectivity between storage nodes.

Replication Pathway

DSA Used: Dijkstra's Algorithm

Finds the minimum latency path for data replication.

Priority Layout

DSA Used: Priority Queue (Min Heap)

Removes the oldest data block first to optimize storage utilization.

COMPLEXITY ANALYSIS

Module	DSA Used	Time Complexity
Catalog Index	Hashing	$O(1)$ Average
Recovery History	Stack	$O(1)$
Buffer Line	Queue	$O(1)$
Registry Index	Binary Search	$O(\log n)$
Line Sorter	Merge Sort	$O(n \log n)$
Connectivity Network	Graph	$O(V + E)$
Replication Pathway	Dijkstra	$O(E \log V)$
Priority Layout	Priority Queue	$O(\log n)$

Where:

- V = Number of Nodes
- E = Number of Connections

SCREENSHOTS OF EXECUTION

Main Menu

```
hey@hey.nandeni@Nandeni-MacBook-Air DSA FINAL % ./dsa
```

```
=====
DISTRIBUTED MEMORY GRID CLUSTER
=====
1. Catalog Index
2. Recovery History
3. Buffer Line
4. Registry Index
5. Line Sorter
6. Connectivity Network
7. Replication Pathway
8. Priority Layout
9. Exit
```

1. Catalog Index – Add/Search Record

5. EXIT

Enter Choice: 1

1. Add Record
 2. Search Record
 3. Display Catalog
- 1

Enter Memory Key: 1001
Enter Data Name: UserProfile
Enter Node Name: Node1

Record Added Successfully!

2. Recovery History – Stack Operations

Enter Choice: 2

1. Add Operation
2. Recover Operation
3. Display History

1

Enter Write Operation: UserProfile
Operation Added Successfully!

```
Enter Choice: 2
```

1. Add Operation
2. Recover Operation
3. Display History

```
2
```

```
Recovered Operation:  UserProfile
```

3.Buffer Line – Queue Processing

```
5. EXIT
```

```
Enter Choice: 3
```

1. Add Request
2. Process Request
3. Display Requests

```
1
```

```
Enter Request: READ UserProfile  
Request Added Successfully!
```

```
Enter Choice: 3
```

1. Add Request
2. Process Request
3. Display Requests

```
1
```

```
Enter Request: WRITE MovieData  
Request Added Successfully!
```

Process Request

```
Enter Choice: 3
```

- 1. Add Request
- 2. Process Request
- 3. Display Requests

```
2
```

```
Processing: READ UserProfile
```

Display Request

```
Enter Choice: 3
```

- 1. Add Request
- 2. Process Request
- 3. Display Requests

```
3
```

```
Pending Requests:  
WRITE MovieData
```

4. Registry Index – Binary Search Output

```
Enter Choice: 4
```

```
Registered Nodes:
```

```
1 2 3 4 5 6 7 8 9 10
```

```
Enter Node ID: 4
```

```
Node Found!
```

5.Line Sorter – Sorted Blocks

```
Enter Choice: 5

1. Add Block
2. Sort Blocks
3. Display Blocks
1
Enter Block Name: BlockC
Enter Compression Benefit: 20
Block Added Successfully!
```

Sort Blocks

```
Enter Choice: 5

1. Add Block
2. Sort Blocks
3. Display Blocks
2
Blocks Sorted Successfully!
```

Display Blocks

Enter Choice: 5

1. Add Block
 2. Sort Blocks
 3. Display Blocks
- 3

Blocks:

BlockB : 80

BlockD : 60

BlockA : 40

BlockC : 20

6.Connectivity Network – Graph Representation

```
Enter Choice: 6

1. Add Connection
2. Display Network
1
Enter First Node: 2
Enter Second Node: 4
Connection Added Successfully!
```

Display Network

```
Enter Choice: 6

1. Add Connection
2. Display Network
2

Connectivity Network:
Node 1 -> 2 3
Node 2 -> 1 4
Node 3 -> 1 4
Node 4 -> 2 3
```

7.Replication Pathway – Shortest Path Result

```
Enter Choice: 7

1. Add Link
2. Find Shortest Path
1
Enter Source Node: 2
Enter Destination Node: 4
Enter Latency: 8
Link Added Successfully!
```

Find shortest Path

```
Enter Choice: 7

1. Add Link
2. Find Shortest Path
2
Enter Source Node: 1
Enter Destination Node: 4
Minimum Latency: 5 ms
```

8.Priority Layout – Eviction Process

Enter Choice: 8

1. Add Block
2. Evict Block
3. Display Blocks

1

Enter Block Name: BlockA

Enter Timestamp: 10

Block Added Successfully!

Display Blocks

Enter Choice: 8

1. Add Block
2. Evict Block
3. Display Blocks

3

Stored Blocks:

BlockA (Timestamp: 10)

BlockB (Timestamp: 20)

BlockC (Timestamp: 30)

Evict Blocks

Enter Choice: 8

1. Add Block
2. Evict Block
3. Display Blocks

2

Evicted Block: BlockA

RESULTS AND FINDINGS

The project successfully demonstrates the implementation of multiple Data Structures and Algorithms within a distributed storage environment. Each module efficiently performs its assigned task while highlighting the importance of selecting suitable algorithms for specific problems.

Hashing enabled efficient memory key management, Stack improved recovery tracking, Queue ensured orderly request processing, Binary Search accelerated node verification, Merge Sort efficiently ranked data blocks, Graphs represented network topology, Dijkstra's Algorithm calculated optimal replication routes, and Priority Queue optimized storage utilization.

CONCLUSION

The Distributed Memory Grid Shard Cluster successfully simulates the core functionalities of a distributed storage system using C++. The project integrates multiple Data Structures and Algorithms into a unified system and demonstrates their practical application in solving storage, networking, searching, sorting, and optimization problems.

The implementation provides a simplified understanding of distributed storage concepts while fulfilling all requirements of Problem Statement 84. It also highlights the importance of efficient algorithm selection in real-world computing environments.

GITHUB REPOSITORY LINK

<https://github.com/Nandeniworks/Distributed-memory-grid-shard-cluster->